Reference

Every file, figure and threshold, in one place. Not a tour; the thing you look something up in.

State. Built and running on Moty's own run. The five invented training years are gone.

The commands

Command	Does	Writes
`prepare`	loads the run, builds the features, cuts them, reports both sizes and the kept list	`data/features.csv`
`train`	splits by time, fits every model and the clerk's checks, reports every figure with its interval	`data/model-report.json`
`score`	one row per invoice for the dashboard; safe to run again over its own output	`03-payables-panel/data/anomalies.csv`
`watch`	walks the run week by week, retrains on a fall or a drift, promotes only on separated intervals; keeps its place	the scored file, `data/watch-state.json`
`serve`	the gauges, on port 9109, re-read every 30 seconds	—

Every command appends one JSON line to `data/runs.log`, with a timestamp that carries its zone. Days of running are only readable through that file.

Leaving it running

The run is history: 2 January to 14 October 2026. `watch` walks it to the end and stops. For the project to keep moving, new invoices have to arrive, and that is the earlier pipeline's job:

Where	Command	Cadence
`01-invoice-pipeline`	`python run.py live --send`	once a day; brings only what was issued since the last run
`01-invoice-pipeline`	`python watch.py --metrics 9108`	left up
here	`python run.py watch`	after each day's intake; carries on from where it stopped
here	`python run.py serve`	left up

Nothing schedules itself. Registering a task that starts these on a timer is a change to the machine, not to this project, and is left to whoever runs it.

The data

One source, and only one: the run the earlier pipeline wrote into `01-invoice-pipeline/Invoices/live/`.

invoices	1,044
PDFs on disk	1,003
errors	37
false positives	48
first invoice	2 January 2026
last invoice	14 October 2026

`labels.csv` carries, per invoice: the error kind or empty, the kind of error it was built to be mistaken for or empty, and `found_after_days`, how long the error took to come to light. Labels arrive late on purpose: a model may only learn from an error already found by its cutoff.

The `erp/` folder holds what the department can check an invoice against: `vendors.csv`, `vendor_changes.csv`, `purchase_orders.csv`, `po_lines.csv`, `contracts.csv`, `receipts.csv`, `shipments.csv`, `rate_card.csv`, `tariffs.csv`, `items.csv`, `ptax.csv` and `approval_policy.csv`.

What is in the run

Error	Real	False positives
a vendor billing with no order behind it	10	2
a freight charge not on the rate card	10	20
a duty at a rate not in force	5	2
a broker fee above its contract	5	6
the same bill again under a new number	3	11
a price above the contract	2	0
more billed than the dock received	2	0
an account or sender the master does not hold	0	6
a round adjustment to the total	0	1
a delivery split under the approval limit	0	0

Three kinds never happened. Two of them exist only as false positives, so the model is asked about them without having seen the real thing.

The features

One row per invoice. Every value is computed from what was knowable on that invoice's own date; a test fails if any of them reads ahead.

Against the contract, the order and the receipt

Feature	What it is
`price_over_contract`	the worst line's price over the agreed price, as a share
`quantity_over_receipt`	the worst line's quantity over what the dock signed for
`receipt_known`	1 when a receipt exists to compare against
`unknown_charges`	freight lines that are neither the move itself nor on the rate card
`known_fixed_fee`	1 when a line matches a fee the rate card names

Against the vendor master

Feature	What it is
`bank_not_on_file`	1 when the account to pay is not one the master holds
`sender_not_on_file`	1 when the sending address is not the vendor's
`vendor_age_days`	days between onboarding and this invoice
`has_order`	1 when an order number is on the page

A bank change announced and recorded before the invoice's due date counts as on file. That is what separates a real change from a fraud.

Against the issuer's own past

Feature	What it is
`same_amount_30d`	1 when the same amount came from this issuer within 30 days
`days_since_last`	days since this issuer's previous invoice
`invoices_30d`	how many this issuer sent in the last 30 days
`amount_zscore`	this total against the issuer's own spread
`same_order_7d_total`	this total plus the same order's invoices in 7 days
`under_limit_but_order_over`	1 when each piece sits under the approval limit and the week's total does not
`same_shipment_7d`	how many of those pieces share one shipment

The digits and the kind

Feature	What it is
`round_thousands`	1 when the total is an exact thousand
`cents_zero`	1 when the total has no cents
`is_freight`	1 for a carrier's bill
`is_customs`	1 for a duty statement or a broker's bill
`grand_total`	the total itself

At customs

Feature	What it is
`duty_rate_off`	how far the duty sits from the rate the NCM carried that day
`fee_over_contract`	the worst broker fee over its contract price
`fee_amended`	1 when the line says the contract was amended

The cut

24 features against 35 errors is far past the usual rule of about ten events per feature. `build.cut` trims the set before any model is trained, on the training rows only, never on the test weeks. It drops features that never move, keeps about one per ten errors, and returns them in the fixed column order. Today that is three.

A retraining inside `watch` is fitted on the same three. Passing no list raises: a silent default is how all 24 crept back in once.

The clerk's checks

Twelve checks a person already makes. Each is a flag; the score is the weighted share of flags raised.

Check	Fires when	Stage
account not on file	`bank_not_on_file`	vendor master
sender not on file	`sender_not_on_file`	vendor master
no order behind a goods bill	no order, not freight, not customs	purchasing
price above contract	over by more than 2%	billing
quantity above receipt	over by more than 2%	receiving
charge not on the rate card	any unknown charge	freight billing
same amount again	within 30 days	billing
order split under the limit	pieces under, total over	billing
round total, not a known fee	exact thousand, no matching fee	billing
young vendor	under 90 days old	vendor master
duty at a rate not in force	off by more than 0.001	customs
broker fee above contract	over by 2%, not amended	customs

How a check earns its weight

`learn_weights(table, cutoff)` measures, on the training rows only, how much each check raises the odds of a real error:

```
weight = ((hits + 1) / (fires + 2)) / base rate
```

The `+1` and `+2` stop a check that fires three times and is right three times from being treated as certainty. A check that never fires weighs nothing. Passing a row dated after the cutoff raises an error.

These checks are **the baseline**. If they beat every model, that is the published result.

How a model is measured

Trained on the earlier weeks, tested on the later ones. The split prints its dates and its error counts on both sides.

Figure	What it is
confusion matrix	caught, missed, false alarm, left alone
precision	of the invoices called wrong, the share that are
recall	of the real errors, the share called wrong
F1	the harmonic mean of the two
average precision	the area under the precision-recall curve
recall by error kind	which kinds are never found

Intervals

About a dozen errors fall in the test weeks. Every figure is reported with an interval around it, computed by resampling. **Two models whose intervals overlap are reported as not separated**, whatever the point estimates say.

Two honesty tests

- A model handed the label as a feature must score near perfect. One that cannot cheat is not learning.
- A model fed shuffled labels must score near the base rate. One that still scores well is reading something it should not.

The loop

One week at a time.

1. The serving model ranks the week; the top of the list is checked.
2. Each check ends confirmed, dismissed, or still open.
3. `feedback.labels_from` turns the verdicts into labels: confirmed is 1, dismissed is 0, never-checked is 0, still open is dropped rather than passed off as clean.
4. The week's precision joins the watch; the drift is measured.
5. A fall or a drift starts a challenger on those labels.
6. The challenger is judged on the weeks since it was trained. A better one takes over. **A tie is not a win.**

Every threshold in this loop — how many weeks judge a challenger, how many weeks the drift looks back on, how far a feature must move, how many weeks under the floor count as a fall — was chosen by Claude and is supported by nothing. They are listed, with that said plainly, in sources.

What the dashboard reads

`score` writes one row per invoice:

Column	Holds
`number`, `supplier`, `kind`, `issue_date`, `week`	who and when
`grand_total`	the amount
`score`	the model's number, 0 to 1
`flagged`	1 for the invoices sent to be checked
`check`, `stage`	the heaviest check that fired, and its stage
`checks`	every check that fired
`decision`	confirmed, dismissed, open, or empty
`origin`	`client` for every flag: the supplier's error, never the reader's

An invoice the model flags that no check explains reads `model only`.

The model's panels go on the cost dashboard, beside the cost figures. Their shape is sketched in panels-sketch.html.

MLflow

Local, SQLite, in `mlflow.db`. The experiment is `invoice-anomalies`.

Thing	Where
the comparison	one parent run, one child per model
the data	logged with each run, by name and hash
the registered model	`invoice-anomaly`
the alias that serves	`@champion`
the one waiting	`@challenger`

A champion built on other feature columns cannot read today's invoices, so its old score is no bar for a new model to clear.

The files

```
02-invoice-anomalies/
├── run.py                      the commands
├── mlflow.db                   the local tracking store
├── docs/
│   ├── tutorial.md             run it, step by step
│   ├── reference.md            this page
│   ├── sources.md              where every number comes from
│   ├── reading-order.md        every file, in order
│   ├── removals.md             invented claims, and what is left of them
│   ├── triage-runbook.md       the work left, step by step
│   └── panels-sketch.html      the four panels, with invented numbers
└── src/
    ├── data/load.py            the run, as typed tables
    ├── features/build.py       the features
    ├── evaluation/
    │   ├── metrics.py          confusion matrix, precision, recall, F1
    │   ├── intervals.py        the interval around every figure
    │   ├── reading.py          perfect data against what was read
    │   ├── splits.py           by time, with late labels
    │   └── watch.py            weekly precision, drift
    ├── models/
    │   ├── rules.py            the clerk's checks and their learnt weights
    │   ├── train.py            the models, recorded in MLflow
    │   ├── lifecycle.py        retrain, challenge, register
    │   ├── scoring.py          the dashboard rows
    │   ├── feedback.py         the verdict as a label
    │   └── exporter.py         the figures the dashboard reads
    └── runs/loop.py            the loop, week by week
```

Known limits

- Every invoice is simulated, and the simulation is known. A result here is a result about the simulation.
- 37 errors is a small number. A time split leaves about a dozen to be tested on, which is why every figure carries an interval.
- The person checking the flags is played by the answer key: a reviewer who never errs and never tires.
- No supervisory or audit standard has been read, and none is met.